



Experiment 8.3

Student Name: Niharika Singh
Branch: BE-CSE
Semester: 5th

UID: 23BCS11624
Date of upload: 27 OCT 2025
Subject Name: FS

1. Aim: Build Role-Based Access Control with Admin, User, and Moderator Roles

2. Objective:

Learn how to implement role-based access control (RBAC) in a Node.js and Express application to restrict access to certain routes or actions based on user roles. This task helps you understand how to store and check user roles in JWT tokens, create flexible authorization logic, and secure sensitive parts of your backend API.

4. Implementation/Code:

```
const express = require('express')  
  
const jwt = require('jsonwebtoken')  
  
const bodyParser = require('body-parser')  
  
const cors = require('cors')  
  
  
const app = express()  
  
app.use(cors())
```

```
app.use(bodyParser.json())
```

```
// Secret key for JWT signing
```

```
const JWT_SECRET = 'supersecretkey123'
```

```
// Sample users with roles
```

```
const users = [  
  { username: 'admin', password: '12345', role: 'Admin' },  
  { username: 'mod', password: '12345', role: 'Moderator' },  
  { username: 'user', password: '12345', role: 'User' }  
]
```

```
// Login route - issues token with role info
```

```
app.post('/login', (req, res) => {  
  const { username, password } = req.body  
  
  const foundUser = users.find(u => u.username === username && u.password  
=== password)
```

```
  if (!foundUser) return res.status(401).json({ message: 'Invalid credentials' })
```

```
  const token = jwt.sign(  
    { username: foundUser.username, role: foundUser.role },  
    JWT_SECRET,  
    { expiresIn: '1h' }  
  )
```

```
  res.json({ success: true, token })
```

```
})
```

```

// Middleware: verify JWT and attach user info
function verifyToken(req, res, next) {
  const authHeader = req.headers['authorization']
  const token = authHeader && authHeader.split(' ')[1]
  if (!token) return res.status(403).json({ message: 'Token missing' })

  jwt.verify(token, JWT_SECRET, (err, decoded) => {
    if (err) return res.status(401).json({ message: 'Invalid token' })
    req.user = decoded
    next()
  })
}

// Middleware: role-based access check
function authorizeRoles(...allowedRoles) {
  return (req, res, next) => {
    if (!allowedRoles.includes(req.user.role)) {
      return res.status(403).json({ message: 'Access denied: insufficient permissions' })
    }
    next()
  }
}

// Routes with role-based protection
app.get('/admin', verifyToken, authorizeRoles('Admin'), (req, res) => {
  res.json({ message: `Welcome Admin ${req.user.username}!`, role: req.user.role })
})

```

```
app.get('/moderator', verifyToken, authorizeRoles('Admin', 'Moderator'), (req, res)
=> {

  res.json({ message: `Welcome Moderator ${req.user.username}!`, role:
req.user.role })

})
```

```
app.get('/user', verifyToken, authorizeRoles('Admin', 'Moderator', 'User'), (req, res)
=> {

  res.json({ message: `Hello ${req.user.username}, this is your user profile!`, role:
req.user.role })

})
```

```
// Public route
```

```
app.get('/', (req, res) => res.send('Role-Based Access Control API is running...'))
```

```
const PORT = 4000
```

```
app.listen(PORT, () => console.log(`Server running on
http://localhost:${PORT}`))
```

5. Output:

JWT Protected Route Demo

Username

Password

Login

Access Protected

Response:

No response yet

Learning Outcome :

You will learn how to use React's useState hook to manage and validate form data, handle user input in real time, and respond to form submission events effectively within a component.